# CHAPTER FOUR: SYSTEM DESIGN AND IMPLEMENTATION

## 4.1 Objective of the Design

The primary objective of designing the Online Campus Waste Management System (OCWMS) is to provide the University of Cross River State (UNICROSS) with a web-based platform that enables students, sanitation staff, and administrators to report, track, manage, and resolve waste incidents across campus.

The system design aims to achieve the following objectives:

1. **Centralised Waste Reporting:** Provide students with an accessible interface to report waste incidents by uploading photographs, selecting waste categories, specifying locations, and setting urgency levels.
2. **AI-Powered Classification:** Integrate artificial intelligence (Google Gemini API) that automatically classifies uploaded waste images into categories (Plastic, Glass, Organic, Paper, Metal, Electronic, Mixed Waste) with handling instructions and recycling guidance.
3. **Role-Based Access Control:** Implement three distinct user roles — Student, Staff, and Administrator — each with tailored dashboards, permissions, and visual themes reflecting their responsibilities.
4. **Real-Time Task Management:** Enable administrators to assign reports to specific sanitation staff, and allow staff to accept, work on, and resolve tasks with photographic evidence of completion.
5. **Interactive Campus Mapping:** Provide a live MapLibre GL map showing campus waste bin locations (with fill levels) and collection vehicles (with operational status) for better resource allocation.
6. **Automated Notifications and Audit Trail:** Generate notifications when reports are submitted, assigned, or completed, and maintain a chronological activity log for accountability and auditing.

7. **Analytics and Reporting:** Provide administrators with visual analytics including charts, KPIs, and trend data for informed decision-making regarding campus sanitation operations.

---

## 4.2 System Architecture

The OCWMS is designed using a **three-tier client-server architecture**, separating the system into the Presentation Tier, the Application Tier, and the Data Tier.

### 4.2.1 Presentation Tier

The user-facing layer implemented as a **Single-Page Application (SPA)** using React 19 with TypeScript. It renders all UI components, captures user input, sends HTTP requests to the Application Tier via the API client, and manages client-side navigation without page reloads. Communication with the server is exclusively through RESTful API calls using the fetch API.

### 4.2.2 Application Tier

The processing layer implemented using **Node.js** with **Express.js**. It receives and processes HTTP requests, executes business logic (authentication, report lifecycle, role verification), performs AI classification via the Google Gemini API, generates automatic notifications and activity log entries, and returns JSON responses. The Application Tier exposes **22 RESTful API endpoints** organised across six route modules:

**Table 4.1: API Route Modules**

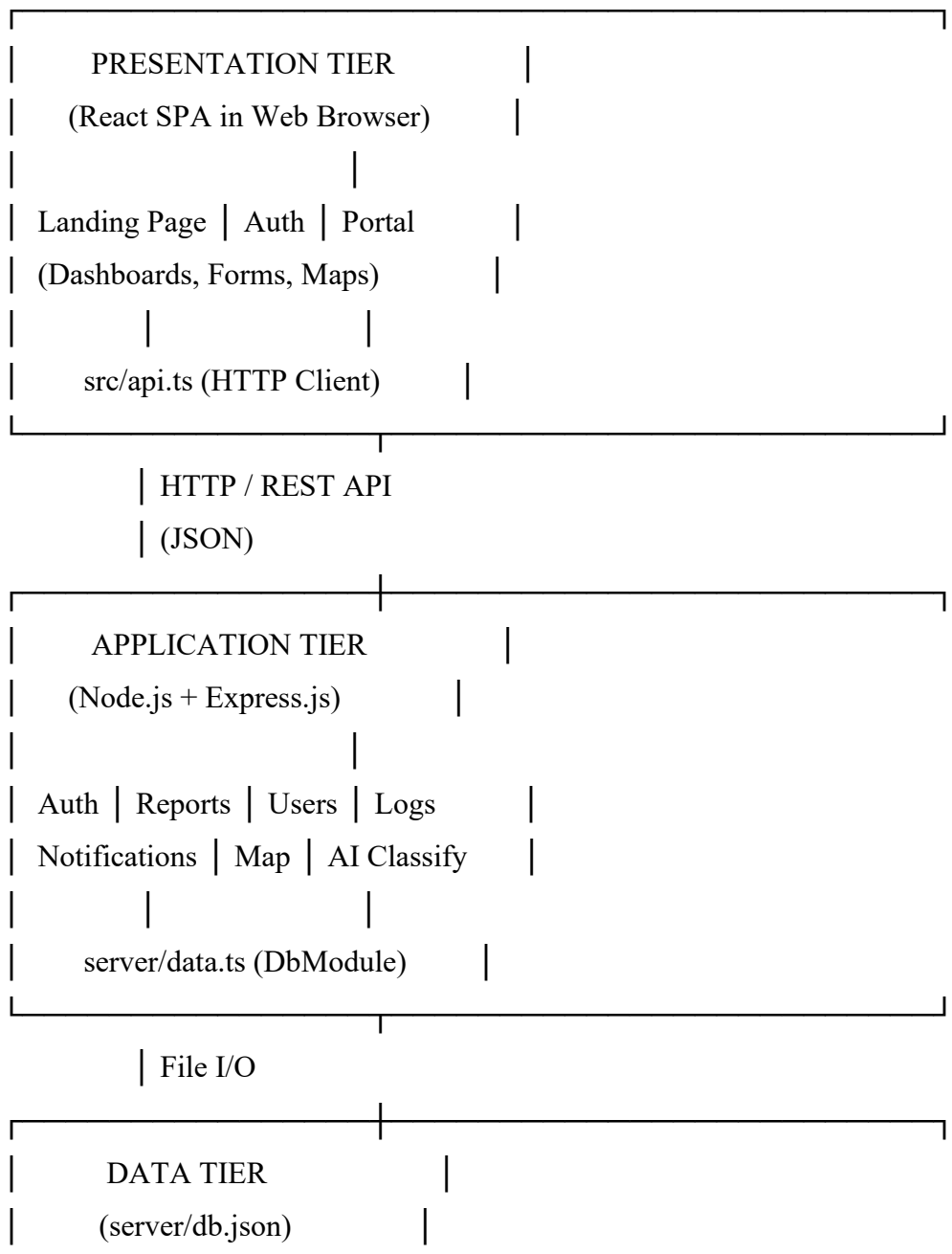
| Module | Prefix | Endpoints | Responsibility |
|---|---|---|---|
| Authentication | /api/auth | 3 | Registration, login, profile retrieval |
| Reports | /api/reports | 6 | CRUD, status updates, assignment, comments |
| Users | /api/users | 5 | Management, role changes, suspension/deletion |
| Notifications | /api/notifications | 3 | Listing and read status management |
| Activity Logs | /api/logs | 1 | Audit trail retrieval |
| Map Data | /api/map | 2 | Bin locations and vehicle data |

Additional endpoints: /api/classify (AI waste classification) and /api/health (system health check).

### 4.2.3 Data Tier

The persistence layer uses a **server-side JSON file** (server/db.json) rather than a traditional RDBMS. The data module (server/data.ts) provides two core functions: getDb() loads the JSON file into memory on first access, and commit() writes the in-memory state back to disk after every write operation. This ensures fast in-memory reads, durable writes, and data recovery after server restarts.

### 4.2.4 Architecture Diagram



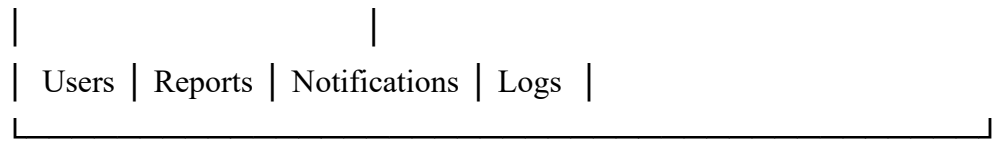



**Figure 4.1: Three-Tier Architecture Diagram of the OCWMS**

## 4.3 Choice of Programming Environment

The programming environment was selected based on suitability for web development, open-source availability, ease of deployment, and community support.

**Table 4.2: Technology Selection and Justification**

| Technology | Version | Purpose | Why Selected | Alternatives Considered |
|---|---|---|---|---|
| React + TypeScript | 19.0.1 / 5.8.2 | Frontend framework | Component reusability, type safety | Vue.js, Angular |
| Vite | 6.2.3 | Build tool / dev server | Fast HMR, native TS support | Webpack, CRA |
| Tailwind CSS | 4.1.14 | UI styling | Utility-first, responsive | Bootstrap, plain CSS |
| MapLibre GL JS | 5.24.0 | Interactive map | Free, no API key, OSM tiles | Google Maps (paid) |
| Recharts | 3.8.1 | Data visualisation | React-native charts | Chart.js, D3.js |
| Lucide React | 0.546.0 | Icon library | Lightweight, consistent | Font Awesome, Heroicons |
| Node.js | 24.16.0 | Server runtime | Event-driven, JS across all tiers | Python/Django, PHP |
| Express.js | 4.21.2 | Web framework | Simple, lightweight, middleware | Koa, Fastify |
| esbuild | 0.25.0 | Server bundler | Sub-second build times | Webpack |

**Frontend technologies:** React was chosen for its component-based architecture promoting code reusability. TypeScript provides static type checking reducing runtime errors. Vite offers fast hot module replacement. Tailwind CSS enables rapid UI development with responsive utilities. MapLibre GL provides free OpenStreetMap tiles without API keys.

**Backend technologies:** Node.js was chosen for its non-blocking, event-driven architecture handling concurrent requests efficiently. Express.js provides simple routing, JSON body parsing, and middleware support. esbuild bundles the server into a single production file.

## 4.4 Database Structure

The OCWMS uses a **file-based database** where all persistent data is stored in server/db.json. The file is managed by server/data.ts, which provides controlled read/write operations.

### 4.4.1 Database Schema

The JSON file contains four top-level collections (arrays):

```
{
  "users":       [ ... ],
  "reports":     [ ... ],
  "notifications": [ ... ],
  "logs":        [ ... ]
}
```

### 4.4.2 Data Access Pattern

All data access follows a consistent pattern through two exported functions:

- **getDb()** — Returns the in-memory copy of the database. On first call, reads server/db.json from disk; subsequent calls return the cached copy.
- **commit()** — Writes the current in-memory state back to server/db.json on disk, called after every write operation.

This ensures fast in-memory reads, durable writes, data recovery after restarts, and minimal disk I/O.

---

## 4.5 Database Table Definitions

Although the system uses a JSON file, the data is structured in tabular form.

### 4.5.1 Users Collection

**Table 4.3: Users Schema**

| Field | Data Type | Constraint | Description |
|---|---|---|---|
| id | String | Primary Key | Unique identifier (e.g., “u_401”) |
| name | String | Required | Full legal name |
| email | String | Required, Unique | University or personal email |
| role | String | Required | “student”, “staff”, or “admin” |

| Field | Data Type | Constraint | Description |
|---|---|---|---|
| status | String | Required | “Active”, “Suspended”, or “Pending” |
| avatar | String | Optional | Profile picture URL |
| matricNumber | String | Optional | Student matric number |
| phoneNumber | String | Optional | Contact phone number |
| faculty | String | Optional | Faculty of study |
| department | String | Optional | Department of study |
| hostel | String | Optional | Hostel allocation |

### 4.5.2 Reports Collection

### Table 4.4: Reports Schema

| Field | Data Type | Constraint | Description |
|---|---|---|---|
| id | String | Primary Key | Unique identifier (e.g., “R-1744”) |
| category | String | Required | Waste category (see Table 4.5) |
| priority | String | Required | “Low”, “Medium”, “High”, or “Emergency” |
| location | Object | Required | Faculty, building, latitude, longitude |
| description | String | Required | Text description of the incident |
| imageUrl | String | Optional | Base64 data URL of uploaded photo |
| completionImageUrl | String | Optional | Photo evidence of completed cleanup |
| status | String | Required | “Pending”, “Assigned”, “In Progress”, or “Completed” |
| dateSubmitted | String (ISO) | Required | Date and time of submission |
| assignedStaffId | String | Optional | ID of assigned staff member |
| assignedStaffName | String | Optional | Name of assigned staff member |
| studentId | String | Required | ID of the reporting student |
| studentName | String | Required | Name of the reporting student |
| comments | Array | Required | Array of Comment objects |
| aiClassification | Object | Optional | AI analysis results |

### Table 4.5: Waste Category Values

| Category | Description | Colour |
|---|---|---|
| Plastic | Bottles, bags, containers | Green |
| Glass | Broken glass, jars, mirrors | Purple |
| Organic | Food waste, leaves, grass | Brown |
| Paper | Paper, cardboard, books | Blue |
| Metal | Cans, aluminium, iron | Grey |
| Electronic | Batteries, bulbs, e-waste | Red |
| Mixed Waste | Unclassified or mixed | Teal |

### 4.5.3 Comments Sub-Schema

### Table 4.6: Comment Object Schema

| Field | Data Type | Description |
|---|---|---|
| id | String | Unique identifier |
| author | String | Name of the comment author |
| authorRole | String | “student”, “staff”, or “admin” |
| content | String | Comment text |
| timestamp | String (ISO) | Date and time posted |

### 4.5.4 Notifications Collection

### Table 4.7: Notifications Schema

| Field | Data Type | Constraint | Description |
|---|---|---|---|
| id | String | Primary Key | Unique identifier |
| title | String | Required | Short notification title |
| message | String | Required | Detailed message |
| timestamp | String (ISO) | Required | Date and time of creation |
| read | Boolean | Required | Whether notification has been read |
| type | String | Required | “success”, “info”, “warning”, or “error” |

### Table 4.8: Notification Generation Events

| Event | Title Format | Type |
|---|---|---|
| Report submitted | “[Category] Incident Reported” | info |
| Staff assigned | “New Dispatch Assignment” | info |
| Report completed | “Cleanup Completed ([ID])” | success |
| User registered | “Welcome to UNICROSS OCWMS!” | success |

### 4.5.5 Activity Logs Collection

### Table 4.9: Activity Logs Schema

| Field | Data Type | Constraint | Description |
|---|---|---|---|
| id | String | Primary Key | Unique identifier |
| action | String | Required | Type of action performed |
| user | String | Required | Name of the user |
| role | String | Required | Role of the user |
| timestamp | String (ISO) | Required | Date and time of action |
| details | String | Required | Detailed description |

### Table 4.10: Activity Log Action Types

| Action | Triggered By | Example Details |
|---|---|---|
| New Report Filed | Student submission | “Filed R-1744 (Plastic) at Block A.” |
| Status: Assigned | Admin assignment | “Marked R-1744 as Assigned.” |
| Status: In Progress | Staff acceptance | “Marked R-1744 as In Progress.” |
| Status: Completed | Staff completion | “Marked R-1744 as Completed.” |
| New Registration | User sign-up | “Registered with Matric U/2021/ENG/0441” |
| User Suspended | Admin action | “Suspended user Bright Okon.” |
| Role Changed | Admin action | “Changed Bright Okon role to staff.” |
| User Deleted | Admin action | “Deleted user Bright Okon.” |

## 4.6 Input and Output Screen Formats

### 4.6.1 Landing Page (Output)

First page displayed on visit. Contains a top navigation bar with logo and “Sign In”/“Get Started” buttons, a hero image carousel (6 slides with trust badges), four feature cards (Garbage Collection Fleet, Smart Waste Bins, Recycling Stations, Trained Personnel), a “How It Works” section (4 numbered steps), animated statistics counters (28,450+ Tons Collected, 12,600+ Routes, 1,580+ Smart Bins, 99% Efficiency), Quick Access login cards (Student/Staff/Admin), and a footer.

### 4.6.2 Login Screen (Input)

Split-panel layout: left panel shows UNICROSS branding with tagline and statistics; right panel toggles between Login and Register forms. Login form includes email input, password field with show/hide toggle, “Remember this session” checkbox, and “Enter Operational Panel” submit button. Three Quick Access buttons (STUDENT, STAFF, ADMIN) enable instant login without credentials.

### 4.6.3 Registration Screen (Input)

Form fields: Full Legal Name, Matric Number, Phone Number, Email Address, Faculty (dropdown with 6 options), Department, Hostel Allocation, Password, and Confirm Password. Submit button: “Create My Ecosystem Profile”.

### 4.6.4 Student Dashboard (Output)

Dark gradient welcome banner with student name and “Report Waste Spot” button. Four KPI cards (Total Submissions, Awaiting Staff, Active Cleanup, Resolved Issues). Weekly Submissions bar chart (Mon–Sun). Waste Categories pie chart (donut). Recent Submissions table with columns: Filing ID, Incident Details (thumbnail + category), Faculty Landmark, Urgency (colour badge), Status (colour dot), Action button.

### 4.6.5 Staff Dashboard (Output)

“SANITATION FORCE DISPATCH” banner with staff name. Four KPI cards (Assigned Tasks, Completed Resolutions, Average Completion Time, Operational Tier). Task table with columns: Task ID, Incident Assets (thumbnail + category), Landmark Coordinates,

Priority badge, Duty Status (animated dot), Action buttons (“Accept Task”, “Resolve Task”, or “TASK RESOLVED” badge).

#### 4.6.6 Admin Dashboard (Output)

Five KPI cards (Total Users, Field Crew, Total Reports, Resolutions, Active Trucks). Three sub-tabs: (1) Sanitation Analytics — area/pie/bar charts, faculty rankings, recent report cards; (2) User Directory — search, role filter, inline role/status editing, delete; (3) System Audit Logs — chronological list with colour-coded action dots.

#### 4.6.7 Report Submission Form (Input)

Photo upload zone with drag/click-to-upload and four preset sandbox photos. AI Classification Panel showing loading spinner, then category, confidence %, handling tip, recycle potential, and green tip. Form fields: Waste Category (dropdown), Incident Urgency (4 segmented buttons), Campus Faculty/Area (dropdown), Building Name, Description. Submit button: “Dispatch Incident Report”. Interactive mini-map on the right with clickable landmarks and GPS coordinates.

#### 4.6.8 Live Fleet Map (Output)

MapLibre GL map with OpenStreetMap tiles, centered on UNICROSS campus [8.351, 4.9755]. Toggle between “Waste Bins” and “Collection Vehicles”. Bin view shows 10 coloured circle markers with fill-level bars (green/amber/red). Vehicle view shows 3 truck markers with driver and status info. Dynamic legends for each view.

#### 4.6.9 Notifications Screen (Output)

“Active Circular Notifications” header with “Mark All as Read” button. Notification cards show coloured dot (green/blue/red), timestamp, title, message, and “Unread” badge. Empty state shows bell icon with “No new alerts”.

#### 4.6.10 Settings Screen (Input/Output)

Profile Settings (avatar, name, email, phone, hostel, save button). Notification Preferences (3 toggle checkboxes). Security Credentials (current/new/confirm password, update button). Green Code of Conduct guidelines.

#### 4.6.11 History and Archives (Output)

Export buttons (PDF/Excel). Filter bar (search, category, status dropdowns). Results table: ID, Material (thumbnail + category), Location, Urgency, Status, Action (“View Timeline”).

---

## 4.7 Program Algorithms

### 4.7.1 User Registration

ALGORITHM RegisterUser

INPUT: name, email, role, matricNumber, faculty, department, hostel

OUTPUT: User record or error

BEGIN

1. Validate email is not empty
2. IF email exists in users collection THEN
   RETURN error "Email already registered"
3. Generate unique user ID ("u_" + random number)
4. Create user record with status="Active" and default avatar
5. Add to users collection (prepend)
6. Create welcome notification and activity log entry
7. Call commit() to save to disk
8. RETURN user record and token

END

### 4.7.2 User Login

ALGORITHM LoginUser

INPUT: email, role

OUTPUT: User record and token, or error

BEGIN

1. Search users collection for matching email (case-insensitive)
2. IF user found THEN

IF status == "Suspended" THEN RETURN 403 error

RETURN user record and token

3. ELSE (not found)

Auto-create new user with provided email and role

Call commit() to save

RETURN new user record and token

END

**4.7.3 Waste Report Submission**

ALGORITHM SubmitReport

INPUT: category, priority, location, description, imageUrl, studentId, studentName, aiClassification

OUTPUT: New report record

BEGIN

1. Generate unique report ID ("R-" + 4-digit number)
2. Set status="Pending", dateSubmitted=current ISO timestamp
3. Create report record with all fields and empty comments array
4. Add to reports collection (prepend)
5. Create notification: "[Category] Incident Reported"
6. Create activity log: "Filed [ID] ([Category]) at [building]"
7. Call commit() to save
8. RETURN report record

END

**4.7.4 Report Status Update**

ALGORITHM UpdateReportStatus

INPUT: reportId, newStatus, completionImageUrl (optional)

OUTPUT: Updated report record

BEGIN

1. Find report by ID; if not found, RETURN 404 error

2. Update report.status = newStatus
3. IF completionImageUrl provided THEN set it on report
4. IF newStatus == "Completed" THEN
   Create notification: "Cleanup Completed ([ID])" for student
5. Create activity log: "Marked [ID] as [newStatus]"
6. Call commit() to save
7. RETURN updated report

END

**4.7.5 Staff Assignment**

ALGORITHM AssignStaff

INPUT: reportId, staffId, staffName, adminName

OUTPUT: Updated report record

BEGIN

1. Find report by ID; if not found, RETURN 404 error
2. Set assignedStaffId, assignedStaffName, status="Assigned"
3. Create notification: "Case [ID] allocated to [staffName]"
4. Create activity log: "Assigned [ID] to [staffName]"
5. Call commit() to save
6. RETURN updated report

END

**4.7.6 AI Waste Classification**

ALGORITHM ClassifyWaste

INPUT: image (base64), description (text)

OUTPUT: Classification result

BEGIN

1. IF GEMINI_API_KEY is configured THEN
   Send image + classification prompt to Gemini API
   Parse and RETURN JSON result (category, confidence, tips)

2. ELSE (heuristic fallback)
   Search description for keywords:
      "bottle"/"plastic" → Plastic (0.92)
      "glass"/"jar" → Glass (0.90)
      "food"/"organic" → Organic (0.95)
      "paper"/"cardboard" → Paper (0.91)
      "metal"/"iron" → Metal (0.88)
      "battery"/"electronic" → Electronic (0.94)
      No match → Mixed Waste (0.85)
   RETURN result with "Simulated" note
END

#### 4.7.7 User Status Management

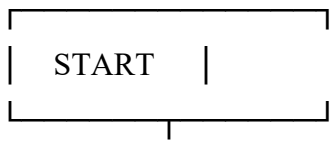
ALGORITHM UpdateUserStatus
INPUT: userId, newStatus, adminName
OUTPUT: Updated user record

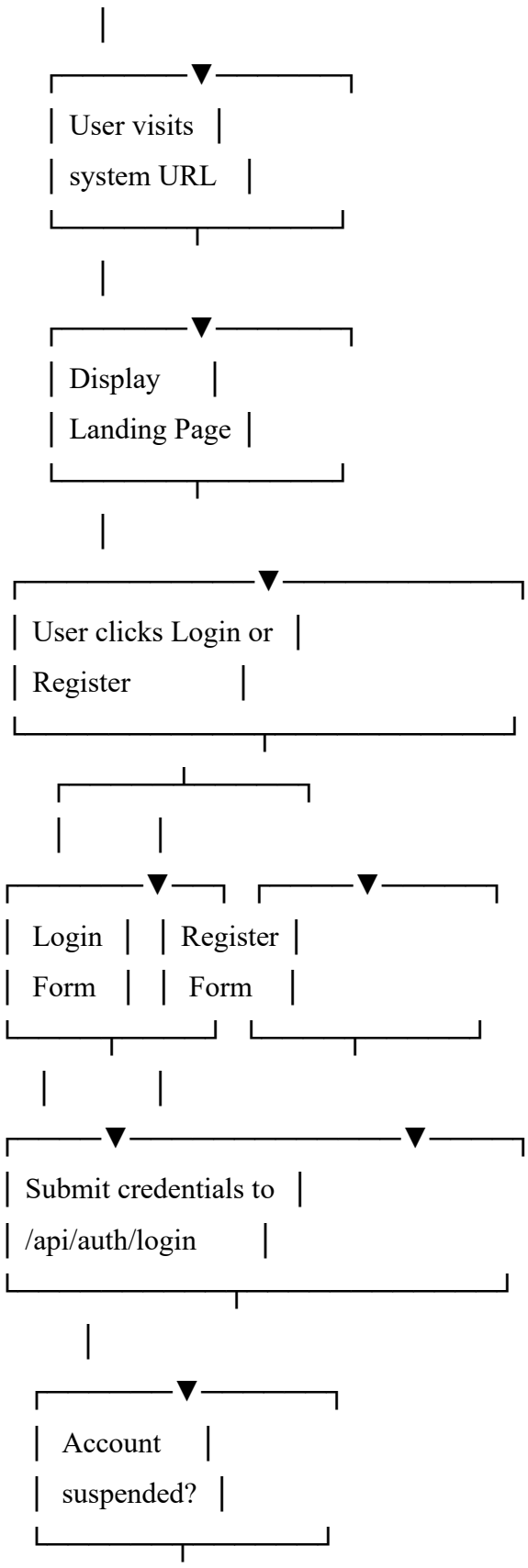
BEGIN
1. Find user by ID; if not found, RETURN 404 error
2. Set user.status = newStatus
3. Create activity log: "[Suspended/Activated] user [name]"
4. Call commit() to save
5. RETURN updated user record
END

---

## 4.8 Program Flowchart

### 4.8.1 Overall System Flowchart




User visits system URL
Display Landing Page
User clicks Login or Register
Login Form
Register Form
Submit credentials to /api/auth/login
Account suspended?

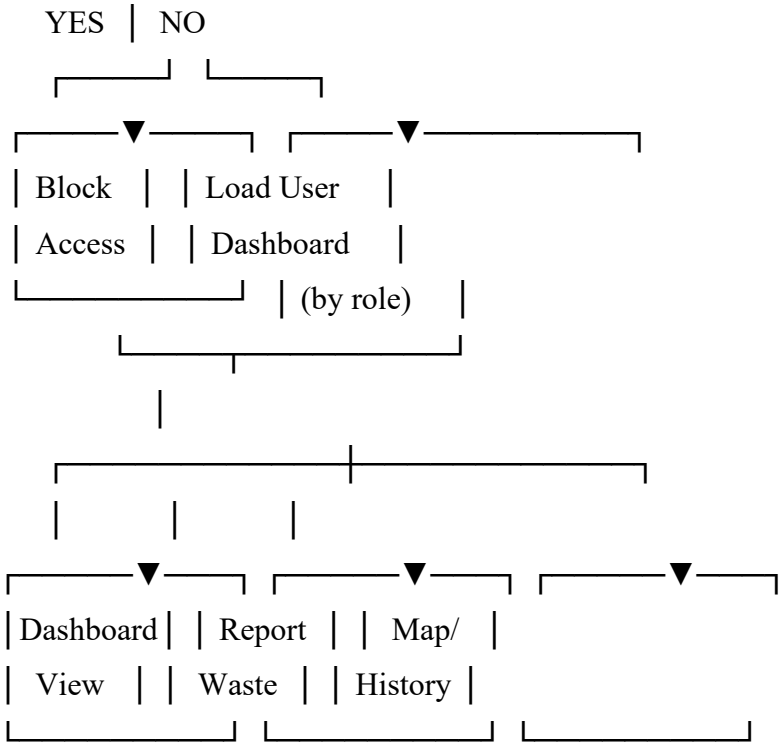
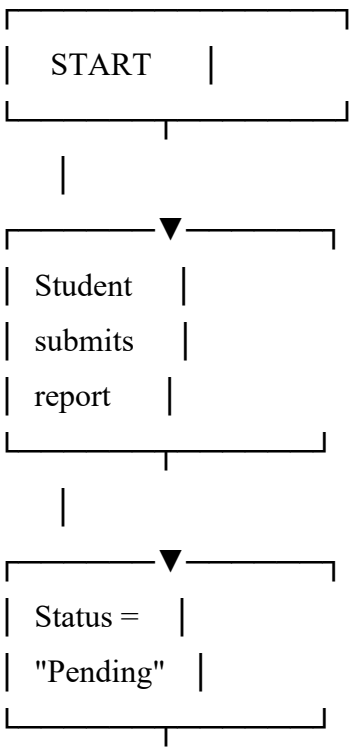
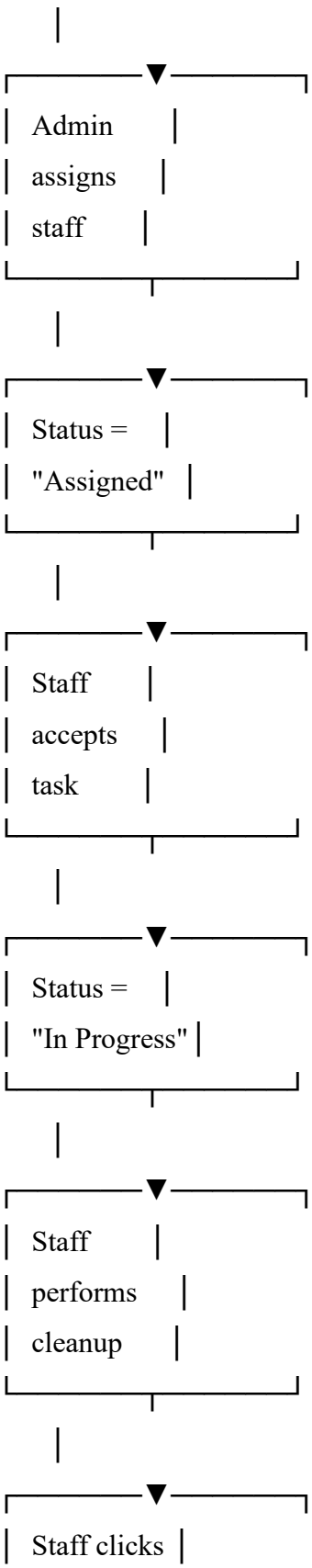



**Figure 4.2: Overall System Flowchart**

### 4.8.2 Report Lifecycle Flowchart




Admin assigns staff
Status = "Assigned"
Staff accepts task
Status = "In Progress"
Staff performs cleanup
Staff clicks

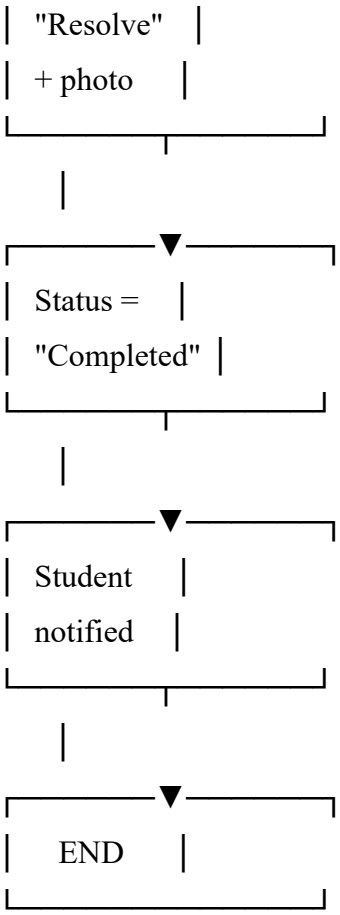



**Figure 4.3: Report Lifecycle Flowchart**

---

## 4.9 Hardware Requirement

### 4.9.1 Development Machine Requirements

**Table 4.11: Hardware Requirements for Development**

| Component | Minimum | Recommended |
|---|---|---|
| Processor | Intel Core i3 (10th Gen) | Intel Core i5 (12th Gen) or higher |
| RAM | 4 GB | 8 GB or higher |
| Storage | 128 GB SSD | 256 GB SSD or higher |
| Display | 1366 x 768 pixels | 1920 x 1080 (Full HD) |
| Internet | Required for npm packages and deployment | Broadband (10 Mbps+) |

### 4.9.2 Deployment/Server Requirements

**Table 4.12: Hardware Requirements for Deployment**

| Component | Minimum | Recommended |
|---|---|---|
| Processor | 1 vCPU | 2 vCPU |
| RAM | 512 MB | 1 GB |
| Storage | 1 GB | 2 GB |
| Network | 100 Mbps | 1 Gbps |

### 4.9.3 End-User Requirements

### Table 4.13: Hardware Requirements for End Users

| Component | Minimum | Recommended |
|---|---|---|
| Device | Desktop, laptop, or smartphone | Desktop or laptop |
| Display | 320 x 480 (mobile) | 1366 x 768 or higher |
| Internet | 3G | 4G/WiFi |
| Browser | Modern browser (Chrome, Firefox, Edge) | Google Chrome (latest) |

## 4.10 Software Requirement

### 4.10.1 Development Software

### Table 4.14: Software Requirements for Development

| Software | Version | Purpose |
|---|---|---|
| Node.js | 24.x+ | JavaScript runtime |
| npm | 10.x+ | Package manager |
| Visual Studio Code | Latest | Code editor |
| Git | 2.x+ | Version control |
| Google Chrome | Latest | Testing and debugging |

### 4.10.2 Runtime/Deployment Software

### Table 4.15: Software Requirements for Running the System

| Software | Version | Purpose |
|---|---|---|
| Node.js | 24.x+ | Server runtime |
| npm | 10.x+ | Production dependency installation |
| Docker | Latest (optional) | Containerised deployment |
| Web Browser | Chrome, Firefox, Edge | Accessing the system |

### 4.10.3 External Services

### Table 4.16: External Service Dependencies

| Service | Required? | Purpose | Cost |
|---|---|---|---|
| Google Gemini AI API | Optional | AI waste classification | Free tier |
| OpenStreetMap Tiles | Yes (auto) | Map rendering | Free |
| Unsplash Images | Yes (embedded) | Profile avatars | Free |

### 4.10.4 Environment Configuration

The system uses a .env file for optional configuration:

GEMINI_API_KEY= # Optional: AI classification

PORT=3000 # Optional: Server port

### 4.10.5 System Startup

1. Install dependencies: npm install
2. Start development server: npm run dev
3. Open browser: http://localhost:3000

For production: 1. Build: npm run build (creates optimised dist/ folder) 2. Start: npm start (runs node dist/server.cjs)

---

## 4.11 Documentation

### 4.11.1 Student User Guide

| Step | Action | Description |
|---|---|---|
| 1 | Register | Click “Get Started”, fill personal details, select faculty |
| 2 | Login | Enter email, click STUDENT Quick Access button |
| 3 | View Dashboard | View statistics, charts, and recent reports |
| 4 | Report Waste | Click “Report Waste”, upload photo, click “Analyse”, fill form, submit |
| 5 | Track Reports | View “My Submissions” for status updates |
| 6 | View Map | Click “Live Fleet Map” for bin/vehicle locations |

### 4.11.2 Staff User Guide

| Step | Action | Description |
|---|---|---|
| 1 | Login | Enter email, click STAFF Quick Access button |
| 2 | View Tasks | View assigned reports in task table |
| 3 | Accept Task | Click “Accept Task” to begin work |
| 4 | Resolve Task | Click “Resolve Task”, select completion photo, submit |

### 4.11.3 Administrator User Guide

| Step | Action | Description |
|---|---|---|
| 1 | Login | Enter email, click ADMIN Quick Access button |
| 2 | View Analytics | Review KPI cards, charts, and trends |
| 3 | Assign Staff | Click report, select staff from dropdown, dispatch |
| 4 | Manage Users | Search, suspend, activate, change roles, delete |

| Step | Action | Description |
|---|---|---|
| 5 | View Logs | Review all system activities in audit trail |

### 4.11.4 Quick Access Profiles (Demo Mode)

For demonstration purposes, three pre-configured profiles allow instant login:

**Table 4.17: Quick Access Profiles**

| Profile | Role | Email | Access Level |
|---|---|---|---|
| Student Resident | Student | brightokon444@gmail.com | Submit reports, view submissions, view map |
| Sanitation Field Staff | Staff | emeka.obi@unicross.edu.ng | View/accept/resolve assigned tasks |
| System Administrator | Admin | florence.effiong@unicross.edu.ng | Full system access, user management, analytics |

### 4.11.5 System Administrator Guide

**Server Deployment:**

1. Clone repository: git clone https://github.com/RealBright1989/UOWMS.git
2. Install dependencies: npm install
3. (Optional) Configure .env with GEMINI_API_KEY
4. Build: npm run build
5. Start: npm start
6. Access: http://localhost:3000

**Docker Deployment:**

1. Build image: docker build -t ocwms .
2. Run: docker run -p 3000:3000 ocwms

**Data Backup:** - Database file: server/db.json - Backup: copy the file - Restore: replace file and restart server

### 4.11.6 Project Structure

UOWMS/

```
├── src/                # Frontend source code
│   ├── api.ts          # REST API client
│   ├── App.tsx         # Root component
│   ├── components/     # React components
```

```
│   └── types/index.ts     # TypeScript definitions
├── server/                # Backend source code
│   ├── data.ts            # Database access module
│   ├── db.json            # Persistent data store
│   └── routes/            # API route handlers
├── server.ts              # Express server entry point
├── Dockerfile             # Docker configuration
└── package.json           # Dependencies and scripts
```

### 4.11.7 Available Scripts

| Command | Description |
|---|---|
| npm run dev | Development server with hot reload |
| npm run build | Production build |
| npm start | Production server |
| npm run lint | TypeScript type checking |